

开发过程与 AI workflow（提交说明）

本项目将“AI 圆桌讨论”拆为可独立验证的边界：用户议题 → 数据/接口 → 专家阵容 → 主持人与专家编排 → 实时事件 → 洞察与总结 → 页面。实际开发使用 Codex 辅助分析、写代码和测试；应用运行时由后端调用 DeepSeek API。题面提到 Claude Code 与 DeepSeek V4 Pro，但本次并未使用 Claude Code，也没有把默认 `deepseek-chat` 宣称为 V4 Pro。workflow 和 Git 历史按真实过程说明；另附的专业 Prompt 案例是事后复盘重构，不冒充未发生的逐字对话。

先从截图抽取功能与交付约束，写设计规格和十项计划，再按测试驱动执行。每个关键边界先写一个会失败的测试：例如阵容数量必须是“1 主持人 + N 专家”，创建议题必须 trim 并限制长度，模型错误不得把讨论置于不可恢复状态。实现后分别运行后端、前端单元测试，再用假网关跑浏览器完整流程。Codex 在开发阶段辅助工程实现；DeepSeek 只在运行阶段生成人物、意图、发言、主持转场、洞察和总结。Key 只在后端环境变量，浏览器没有输入框或隐藏推理字段。

三个难题最能说明工程拆解。第一，专家不是固定轮流发言：各专家并行依据相同上下文提出是否举手与可公开意图，选择器根据相关性、新颖度、紧迫度和公平性选人；只有被选者生成 1-2 句发言，主持人按阶段追问、转场与总结。早期测试通过后，真实浏览器检查仍发现“举手”从未出现在事件流，于是补失败用例并让编排器发布/清除举手状态。

第二，刷新或断线不能让讨论丢失：SQLite 保存消息、洞察和递增事件序号。页面先取完整快照，再从该序号后的 SSE 事件补发；重复事件去重，序号跳跃时重取快照，连续断线后只读轮询。后端重启会把中途任务标为可恢复，用户可从检查点继续。各讨论按 ID 隔离，模型调用由共同的四槽信号量约束。一次浏览器示例运行暴露了假模型把问号议题直接嵌入发言，违反了句数约束；回归测试复现后，假模型清理终止标点并按专家立场生成相反意见。

独立代码审查又找到一个更隐蔽的竞态：快照先读到旧事件序号，却可能读到随后写入的新发言，导致 SSE 补发重复。我们先写并发读写测试复现，再给快照查询加同一读事务，并让前端按消息 ID 二次去重。审查还复现了六专家耗尽 100 次调用、无法总结的情况；改为每轮预留完整调用预算，并对反复失败直至预算耗尽的情况明确停止恢复，不伪装成讨论完成。六专家场次、总结重试和持续请求失败都用测试覆盖。

第三，“幻觉检测”被限定为证据风险而非事实裁判：模型标记与真实发言绑定的原句、原因和状态，后端过滤无效 ID 与不属于发言的引用。提示可点回原发言；即使审查失败，发言仍保留并标记审查不可用。当前没有外部检索或权威数据库，因而“需外部核实”不等于已核实，也不能据此自动宣称真假。

验收分为两层。假网关用稳定、可复现的输出检查 UI、并发隔离、失败恢复、SSE 和风险定位；真实 DeepSeek 冒烟单独检查外部 API 响应、结构化输出、耗时及总结质量，不把假网关结果当成真实模型质量。桌面和 390px 手机截图来自实际浏览器。最终的压缩包、仓库邀请和邮件由用户核对并发送；本项目不会自动外发。